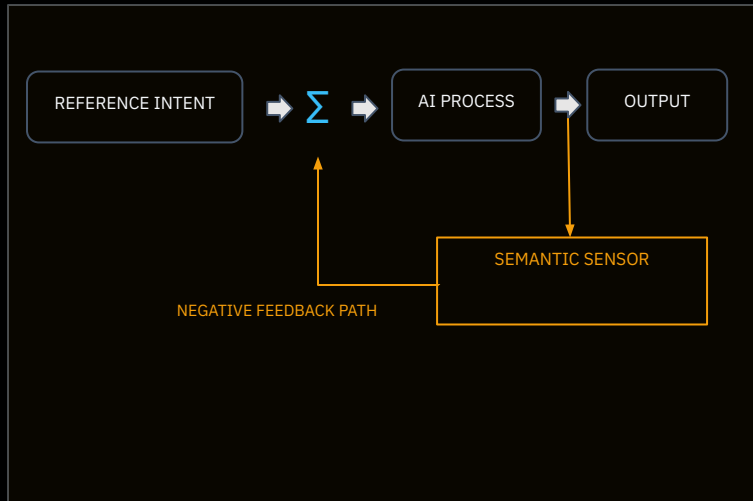


# Designing Non-Deterministic Systems

Maintaining engineering rigor in the AI era through tolerance systems, barriers, and lifecycle control for stochastic software.



## NOT AN API INTEGRATION

LLMs in industrial software change the engineering object itself.

## FROM LINES TO DISTRIBUTIONS

Design moves from linear behavior to probabilistic output envelopes.

## CONTROL THEORY RETURNS

Stable AI systems require closed-loop measurement and correction.

# Beyond the AI Hype

The engineering question is not whether models are conscious. It is what happens when cognition-like operations become an external runtime dependency.



## Writing

### EXTERNALIZED MEMORY

Human memory moved from fragile biological recall into durable external symbols.



## Computers

### EXTERNALIZED CALCULATION

Mathematical operations moved into machines with deterministic execution.



## Large Models

### EXTERNALIZED COGNITION

Reasoning-like token sequences are generated dynamically for each context.

### KEEP SKEPTICISM

Healthy engineering skepticism is the first validation instrument against marketing mythology.

### NO AGENCY CLAIM

Models do not possess will, consciousness, or grounded understanding of the world.

### OPERATIONAL SHIFT

Developers now design the environment in which behavior is generated at runtime.

# A World Where Variance Is Not Zero

SOFTWARE 1.0 · DETERMINISTIC CODE

$$y = f(x)$$

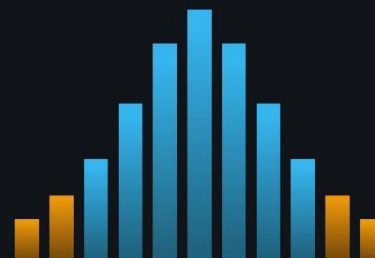
Variance  $\sigma^2 = 0$

## OLD CONTRACT

The same input must always produce the same output.

SOFTWARE 2.0 · STOCHASTIC SOFTWARE

$$y \sim P(y|x)$$



LONG TAIL

USEFUL DISTRIBUTION

LONG TAIL

## NEW MATERIAL

Each call samples from a distribution of possible responses.

## ENGINEERING TASK

Do not destroy variance; shape and constrain its useful zone.

# The New Definition of a Bug

In behavioral software, correctness is no longer a single expected output. It is an approved operating band inside a probabilistic distribution.

## SOFTWARE 2.0 DEFECT MODEL

A bug is a statistical excursion beyond approved business tolerances.

### CLASSIC DEFINITION BREAKS

The code may be unchanged while model behavior still becomes unacceptable.

### SYNTAX IS NOT ENOUGH

The system can be technically valid and still semantically wrong.

### NEW DIAGNOSIS

Engineers inspect drift and distribution deformation, not only broken code paths.

OUTPUT PROBABILITY DISTRIBUTION

BUSINESS TOLERANCE ENVELOPE

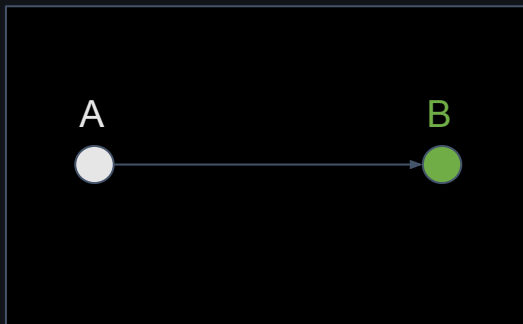


**Release criterion:** output must remain inside tolerance bounds; tail events become defects.

# Designing the Space of Possibilities

In Software 2.0, a requirement is no longer a script for one path. It is the engineered boundary of the probabilistic core's safe operating area.

## OLD REQUIREMENT MODEL



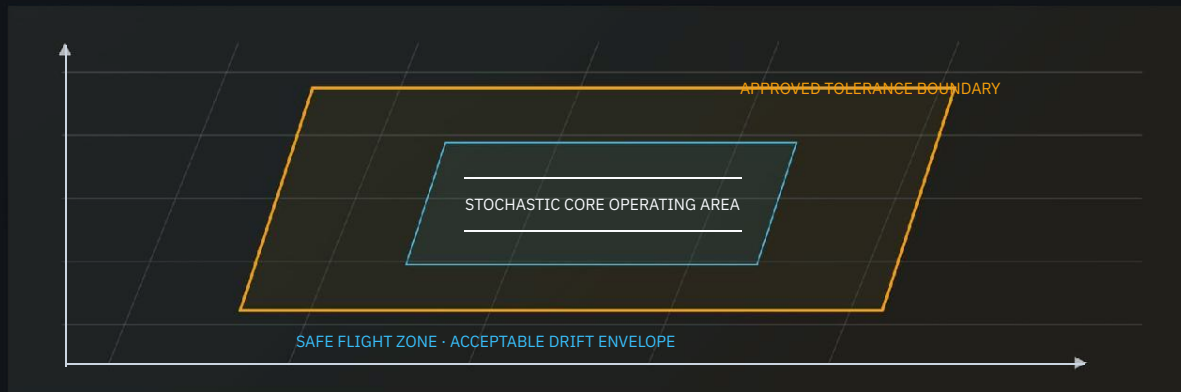
A narrow deterministic rail: button A must always produce window B.

single path · zero drift

## NOT A STEP SCRIPT

Requirements stop enumerating every sentence the system may produce.

## NEW REQUIREMENT MODEL



## DEFINE THE ENVELOPE

Teams specify topics, data sources, constraints, and forbidden regions.

## ENGINEER DRIFT

Each business zone receives explicit limits for acceptable semantic movement.

# From Static Requirements to Statistical Gates

Readiness and completion are no longer proven by a single green run. In stochastic software, the contract must quantify risk, cost, tolerance, and statistical confidence.

## Traditional DoR / DoD

SOFTWARE 1.0

BINARY VALIDATION CONTRACT

|         |                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------|
| READY   | <b>Feature text is specified.</b> The requirement describes expected deterministic behavior and acceptance criteria. |
| COST    | <b>Compute is assumed stable.</b> Runtime cost is rarely part of the readiness contract.                             |
| DONE    | <b>Unit tests are green.</b> Manual verification confirms the happy path and fixed outputs.                          |
| RELEASE | <b>Pass / fail decision.</b> A task is shipped after code review, test pass, and tester approval.                    |

### OLD CHECKPOINT

“The task works on my machine, tests are green, and the manual pass succeeded.”

## Stochastic DoR / DoD

SOFTWARE 2.0

STATISTICAL QUALITY CONTRACT

|         |                                                                                                                        |
|---------|------------------------------------------------------------------------------------------------------------------------|
| READY   | <b>Risk profile is digitized.</b> Business tolerance bounds and prohibited drift zones are defined upfront.            |
| BUDGET  | <b>Token cost is bounded.</b> The feature enters development with a measurable cost envelope.                          |
| DONE    | <b>Eval Gates pass at scale.</b> Large-sample runs prove accuracy, drift, and hallucination metrics are inside bounds. |
| RELEASE | <b>Confidence interval is accepted.</b> Quality is released as a measured distribution, not a one-off observation.     |

### NEW CHECKPOINT

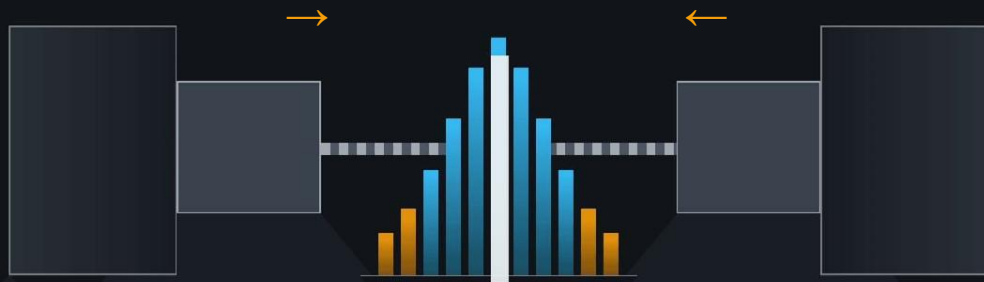
“The output distribution stays within approved business tolerances under automated evaluation.”



# Do Not Crush the Distribution

The traditional reflex is to eliminate uncertainty. In stochastic software, that reflex destroys the very capability the model was introduced to provide.

## MECHANICAL COERCION OF A PROBABILISTIC MATERIAL



DESTRUCTION OF ENGINEERING VALUE

**Goal:** shape the useful zone of the curve.

**Barrier:** cut off tails where critical hallucinations live.

## DIAGNOSIS

### TOTAL-CONTROL REFLEX

Classical teams respond to uncertainty by reducing freedom: temperature zero, rigid regexes, exact matching.

### WRONG TEST OBJECT

Exact-string unit tests treat a probabilistic material as if it were a deterministic function.

### DESTROYED CAPABILITY

If variance is fully suppressed, the cognitive engine becomes a slow, costly, unreliable database.

## WRONG-TOOL METAPHOR



Manually dragging a robot vacuum across the floor: action exists, intelligence is bypassed.

**Principle:** control variance; do not erase it.

# Where AI Creates Engineering Value

Software 2.0 is designed across three connected planes: intent capture, cognitive routing, and constrained contextual output.



**Engineering task:** design these planes as one controlled probabilistic environment, not as isolated prompt tricks or disconnected UX experiments.

INTENT

PLANNING

CONSTRAINTS

# Validators Are Only the Muscles

Schemas, orchestrators, and JSON modes can force form. They cannot determine whether the content is true, safe, or semantically aligned with business intent.

## Syntactic Entropy

VALIDITY OF FORM

### WHAT TOOLS CONTROL

Types, fields, JSON shape, required keys, and parser-compatible structure.

### TYPICAL INSTRUMENTS

Pydantic, Instructor, JSON Mode, Guidance, LangChain, LangGraph.

SCHEMA

PARSER

MODE

### GUARANTEE

The output can be consumed by code without crashing the pipeline.

## Semantic Entropy

TRUTH OF MEANING

### WHAT REMAINS UNCONTROLLED

Factuality, causality, toxicity, hallucination, business intent, and operational safety.

### FAILURE MODES

A perfect object can still contain a fabricated report or dangerous instruction.

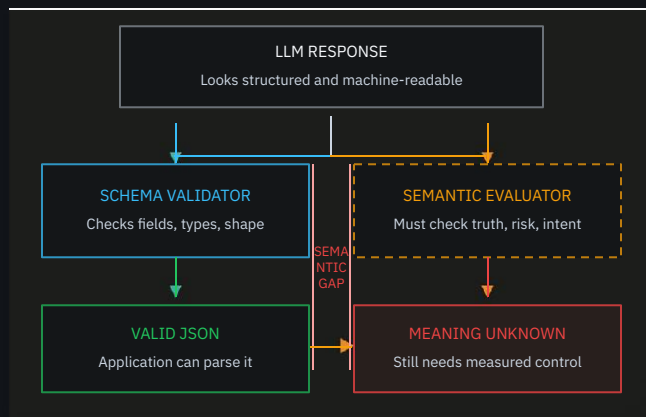
HALLUCINATION

TOXICITY

### MISSING ORGAN

Traditional code sees object structure, but it is blind to internal meaning.

### VALIDATOR PIPELINE VS SEMANTIC CONTROL



**The trap:** a syntactic pass is not a semantic release decision. It only proves the object is executable by code, not acceptable for the business.

### CODE-LEVEL CONTROL

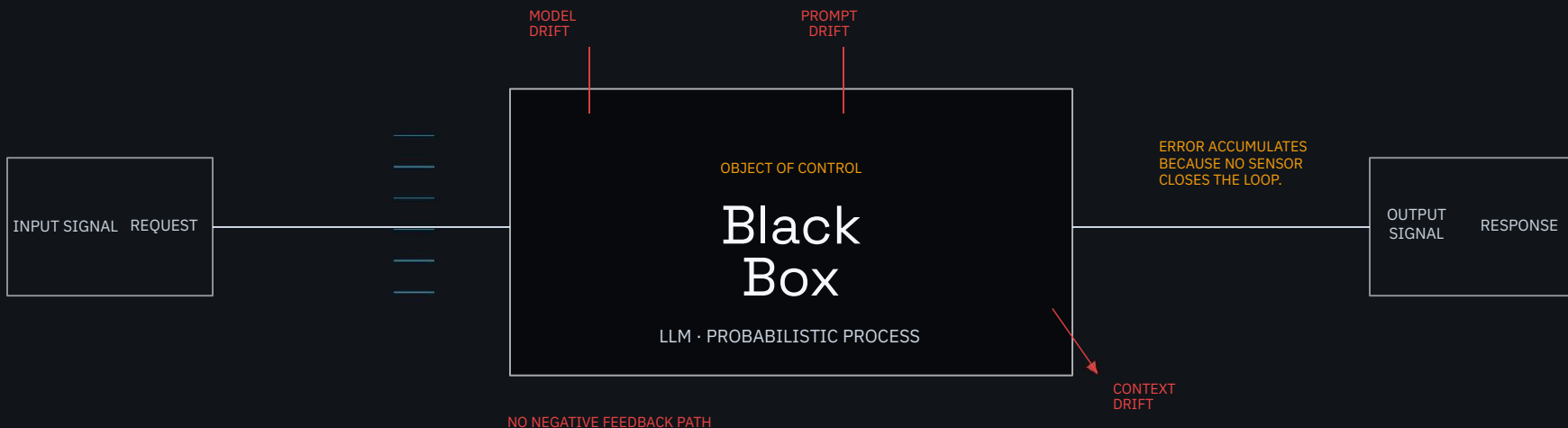
Excellent for structure, serialization, and mechanical execution.

### ARCHITECTURE-LEVEL CONTROL

Required for meaning, judgment, safety, and business-aligned behavior.

# Black Boxes Degrade in Open Loops

Most AI applications ship as open-loop systems: a request enters, a black-box model answers, and the result is released without internal measurement or correction.



## BLACK-BOX OBJECT

The model state is opaque, high-dimensional, and probabilistic from the application's perspective.

## OPEN-LOOP RELEASE

The system hands output to the user before measuring whether it matches business intent.

## DRIFT IS INEVITABLE

Model, prompt, and context changes increase entropy until the system leaves its safe operating band.

# Only Feedback Stabilizes Stochastic Systems

A non-deterministic black box cannot be stabilized by hope, prompt complexity, or subjective review. It requires continuous negative feedback.

## CYBERNETIC THEOREM

Control begins when the system can measure its own error and correct the next input.

## CONSTANT CORRECTION

High-variance systems must be adjusted during operation, not only before release.

## CLOSED LOOP

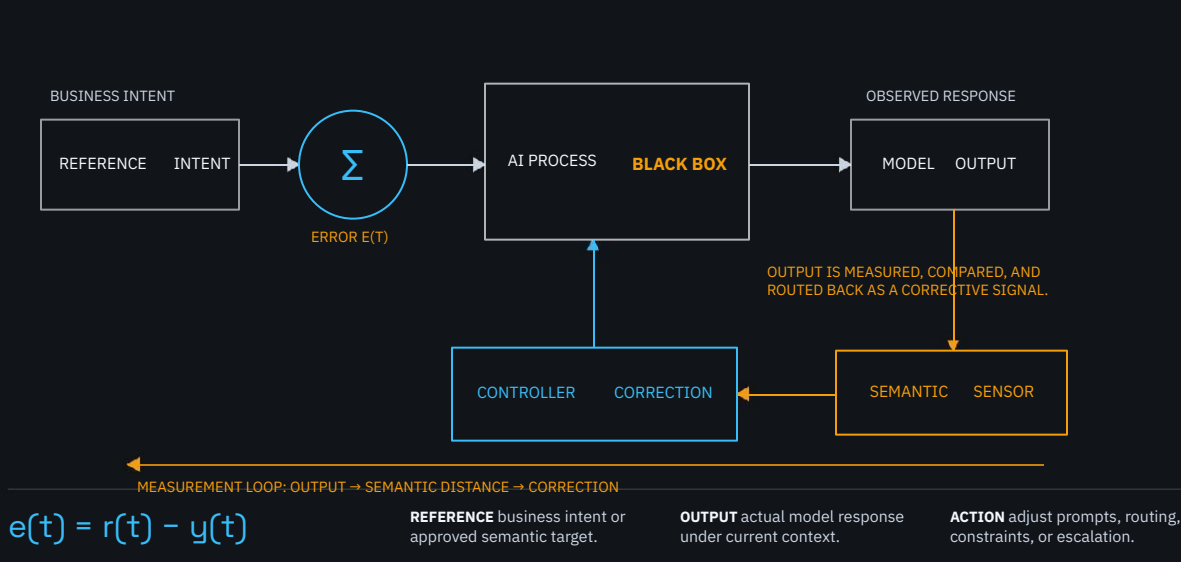
Probability distributions become governable only when the output returns as a signal.

## NO VIBE CHECK

Subjective inspection is replaced by continuous measurement of semantic error.

## CLOSED-LOOP AI CONTROL SYSTEM

## NEGATIVE FEEDBACK PATH



# Anatomy of the AI Controller

Uncertainty Architecture maps classical control theory onto a full software stack for stabilizing stochastic behavior.

FOUR-LAYER CONTROL STACK



REMOVE ONE LAYER AND THE CONTROL LOOP OPENS

SOFTWARE COMPONENT MAPPING

**CONTROLLER**  
**SAFETY** Prompt Registry, semantic Kill Switches, HITL gateways, and runtime policy changes control distribution parameters.

**SENSORS**  
**QUALITY** Golden Sets, Eval Gates, drift metrics, and semantic monitors measure distance from business intent.

**CONSTRAINTS**  
**LIMITS** Strict JSON Schemas, Pydantic validators, and typed interfaces provide mechanical stops for format and structure.

**ACTUATORS**  
**WORK** LangChain, API calls, data pipelines, and tool orchestration perform the physical work on data.

**UA principle:** muscles without nerves and a brain are powerful but blind; control emerges only when execution, limits, measurement, and judgment form one loop.

# Prompt Registry & Versioning

Prompts are not comments near the code. They are behavioral configuration that must be versioned, loaded, observed, and rolled back like any production control surface.

ANTI-PATTERN



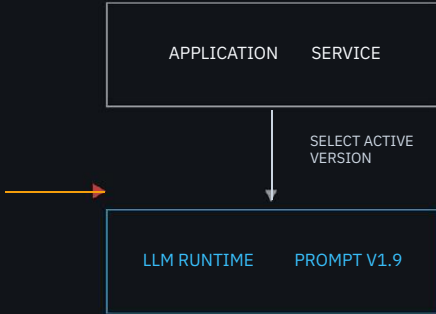
Behavior changes are hidden in deployments, commits, and emergency hotfixs.

SEPARATE BEHAVIORAL PLANE

| Prompt Registry            |                                       |          |
|----------------------------|---------------------------------------|----------|
| RUNTIME SOURCE OF BEHAVIOR |                                       |          |
| V1.8                       | Stable support tone; low-risk answers | ROLLBACK |
| V1.9                       | Current production policy and style   | ACTIVE   |
| V2.0                       | Candidate prompt under Eval Gate      | CANARY   |
| V2.1                       | Blocked after semantic regression     | HALTED   |

PROMPT BEHAVIOR IS GOVERNED OUTSIDE APPLICATION CODE

RUNTIME LOADING



**No redeploy required:** the controller can promote, rollback, or halt behavior through versioned prompt configuration.

VERSIONING

Every behavioral change has an identifier, owner, and review trail.

ROLLBACK

Unsafe drift can be reversed without rebuilding the application.

EXPERIMENT CONTROL

Canaries and A/B runs are tied to explicit prompt versions.

AUDITABILITY

Incidents can be traced to the exact behavioral configuration in force.

# Golden Sets and Eval Gates

AI quality cannot be approved by intuition. It must be measured against calibrated examples, automated semantic checks, and explicit release thresholds.

LABORATORY-GRADE EVALUATION PIPELINE



**GOLDEN SET CONTENTS**  
Inputs, expected semantic properties, unsafe outputs, edge cases, and business-critical scenarios.

ACCURACY / TASK FIT



Must clear the approved minimum quality threshold.

HALLUCINATION RISK



Must remain below the maximum risk boundary.

SEMANTIC DRIFT



Distance from intent must stay inside the tolerance envelope.

EVAL GATE OUTPUT

|  |                                                                      |
|--|----------------------------------------------------------------------|
|  | <b>BLOCK</b><br>Regression or safety failure exceeds tolerance.      |
|  | <b>CANARY</b><br>Metrics are acceptable only under limited exposure. |
|  | <b>RELEASE</b><br>All semantic and operational gates pass.           |

**Gate principle:** deployment is a control action triggered by measurement, not by confidence in a single demo.

OLD APPROVAL MODE

Manual reviewers decide whether the answer “feels good enough” on a few examples.



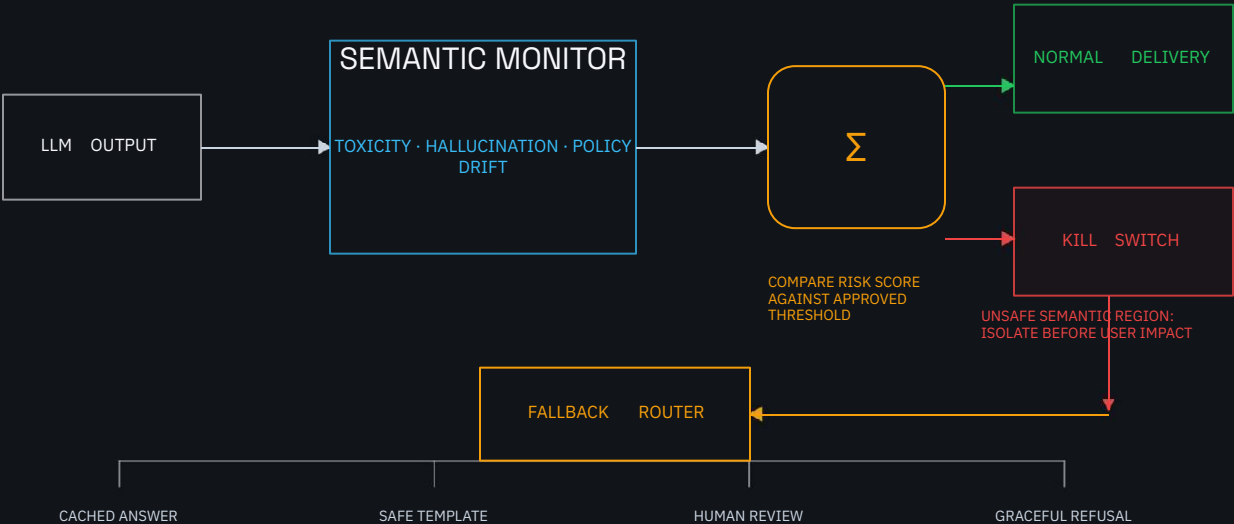
NEW APPROVAL MODE

Automated Eval Gates compare distributions against Golden Sets before each behavioral change ships.

# Designing Stochastic Resilience

A resilient AI system does not assume the model will behave. It continuously monitors semantic risk, isolates unsafe behavior, and routes users into controlled fallback paths.

RUNTIME FAULT-CONTAINMENT FLOW



**MONITORING**

Semantic risk is treated as an operational signal, not a post-release complaint.

**CONTAINMENT**

The kill switch is part of the architecture, not an emergency spreadsheet process.

**CONTINUITY**

Fallbacks protect users while the stochastic core is corrected or bypassed.

RESILIENCE CONTRACT

The controller must fail into safety, not into silence.

**MEASURE**

Every high-risk output passes through semantic sensors before release.

**INTERRUPT**

Threshold breaches automatically stop delivery and isolate the unsafe path.

**RECOVER**

Fallback flows preserve service continuity while reducing risk exposure.

**LEARN**

Incidents update Golden Sets, prompts, thresholds, and escalation policy.

# System Boundary States: Architectural Veto and Systemic Risk

System constraints, economic volatility, and safety boundaries define the point where a stochastic system should not be built today.

THREE SYSTEMIC PRESSURE VECTORS

**Latency Overhead**

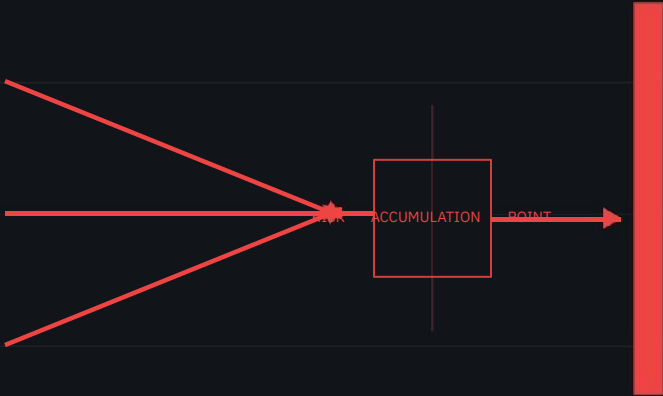
Critical delay introduced by control loops, semantic monitors, and escalation paths.

**Token Cost Volatility**

Vendor pricing instability can destroy unit economics after release.

**Black Box Weight Shift**

Provider-side model updates may change behavior without local code changes.



BOUNDARY OF ENGINEERING RIGOR

ТОЧКА АРХИТЕКТУРНОГО ВЕТО

BLEEDING EDGE

Operations may leave deterministic software practice and enter research-grade uncertainty.

ECONOMIC GATE

Token cost caps and business value become part of the release contract.

VETO CONTRACT

Engineering rigor includes the discipline to say: **do not build this AI path.**

SYSTEM BOUNDARY

Some work sits between classical engineering and live scientific research.

SOFTWARE 2.0 RISK

Cost and behavior may depend on vendors, hidden weights, routing, and pricing surfaces.

ARCHITECTURAL VETO

The architect must declare when the safety boundary cannot be crossed.

SAFETY LIMIT

Do not force AI into workflows where no reliable deterministic path exists.

# PM: From Story Owner to Distribution Economist

In AI products, the PM no longer owns only “what should the model answer?” The PM owns which output distributions, error rates, and cost surfaces are acceptable for the business.

## ROLE TRANSFORMATION

BEFORE

### Story Owner

Specifies visible user stories, acceptance text, and expected flows.



AFTER

### Distribution Economist

Defines business tolerances for probabilistic behavior, cost, escalation, and liability.

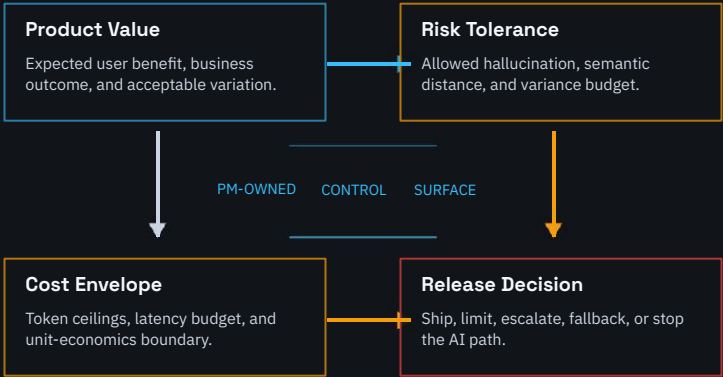
## CORE PM QUESTIONS

- What distributions are acceptable?
- What error rate is economically tolerable?
- When does AI output become liability?

## NEW ROLE DEFINITION

The PM becomes an operator of tolerances, not just a writer of requirements.

## TOLERANCE OPERATING CHAIN



The PM translates stochastic behavior into a measurable product contract: value only exists while the distribution remains inside the approved business envelope.

## ACCEPTANCE CRITERION

Release quality is judged by distribution behavior, not a single successful demo output.

## PM-OWNED TOLERANCES

|                                                                             |       |
|-----------------------------------------------------------------------------|-------|
| <b>HALLUCINATION TOLERANCE</b><br>Maximum fabricated or unsupported claims. | LIMIT |
| <b>ESCALATION THRESHOLD</b><br>When uncertainty must route to humans.       | HITL  |
| <b>SEMANTIC DISTANCE</b><br>Allowed deviation from business intent.         | BAND  |
| <b>TOKEN-COST CEILING</b><br>Budget boundary per product flow.              | COST  |
| <b>VARIANCE BUDGET</b><br>Acceptable spread of behavior over time.          | RANGE |
| <b>FALLBACK BEHAVIOR</b><br>User/business path when AI is unsafe.           | STOP  |

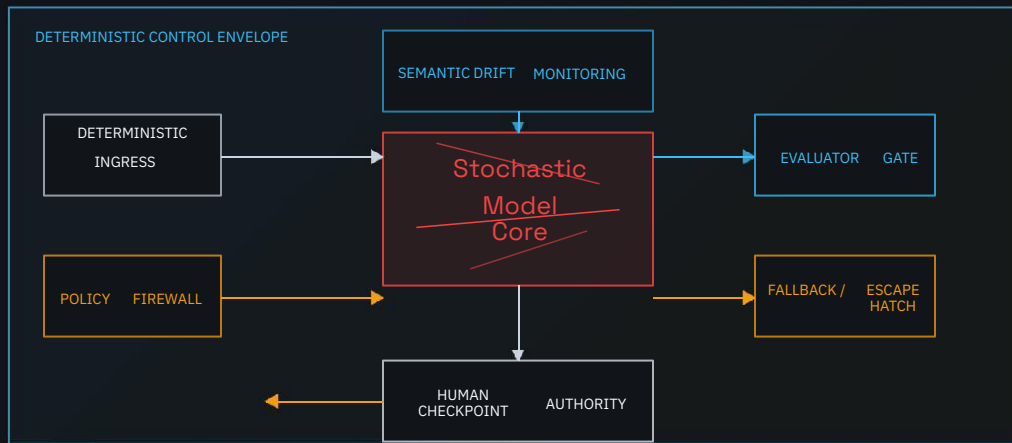
## BUSINESS BOUNDARY

AI stops being product value when its uncertainty exceeds the economic or liability envelope.

# Architect: From System Designer to Systems Risk Engineer

The architect does not merely connect AI components. The architect designs deterministic boundaries around a stochastic core, including rollback, fallback, provider abstraction, observability, and human intervention.

## DETERMINISTIC ENVELOPE AROUND STOCHASTIC BEHAVIOR



MODEL-PROVIDER ABSTRACTION: ISOLATE VENDORS, WEIGHTS, ROUTING, AND DEPLOYMENT CHANGES.

ROLLBACK AND ARCHITECTURAL VETO REMAIN OUTSIDE THE STOCHASTIC CORE.

## ARCHITECT OWNERSHIP

Architectures must define where non-determinism is allowed to exist.

### ISOLATION BOUNDARIES

Confine probabilistic behavior behind deterministic interfaces.

### ROLLBACK PATHS

Return safely when models, prompts, or providers drift.

### PROVIDER ABSTRACTION

Prevent vendor internals from becoming product architecture.

### DETERMINISTIC ESCAPE HATCHES

Fallback to safe behavior when uncertainty is excessive.

### SEMANTIC DRIFT OBSERVABILITY

Make behavioral change visible, measurable, and auditable.

### HITL CHOREOGRAPHY

Define who intervenes, when, with what context, and with what authority.

## WEAK ARCHITECTURE

A pipeline pushes stochastic outputs forward and asks a human to approve when things look uncertain.



## STRONG ARCHITECTURE

The architect designs boundaries around non-determinism, not just pipelines through it.

# From Binary Tests to Distributions

QA no longer asks only whether one expected output appeared. QA calibrates the sensors that determine whether model behavior remains statistically inside the approved semantic envelope.

ROLE TRANSFORMATION

BEFORE

## Binary Tester

Checks deterministic steps, verifies fixed outputs, and marks a case pass or fail.



AFTER

## Sensor Calibrator

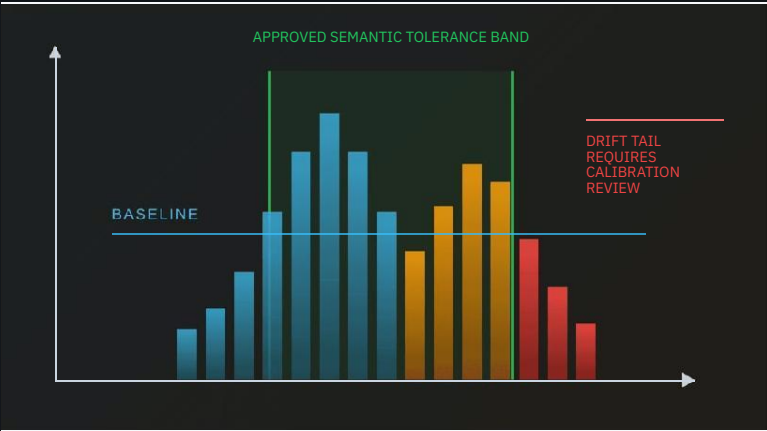
Maintains Golden Sets, tunes semantic metrics, and validates drift statistically.

**QA artifact shift:** from test cases as scripts to calibrated datasets as measurement instruments.

TEST OBJECTIVE

Measure the output distribution, not only one sampled response.

SEMANTIC DRIFT VALIDATION INSTRUMENT



■ GOLDEN BASELINE      ■ CURRENT RUN      ■ OUT-OF-BAND

**Evidence standard:** release decisions are based on distribution behavior across Golden Sets, not a single successful answer.

QA INSTRUMENT

Golden Sets become the calibration standard for semantic sensors.

RELEASE EVIDENCE

Eval Gates require statistical proof that drift remains within tolerance.

QA-OWNED CONTROL ARTIFACTS

GS

**GOLDEN SETS**

Representative cases, adversarial prompts, edge inputs, and expected semantic properties.

EG

**EVAL GATES**

Automated pass criteria for quality, risk, regression, and tolerance boundaries.

DR

**DRIFT REVIEW**

Statistical comparison between baseline and current model behavior.

CI

**CONFIDENCE**

Release evidence includes sample size, uncertainty, and repeatability.

IR

**FEEDBACK LOOP**

Incidents update datasets, thresholds, prompts, and resilience policy.

**INCIDENT LEARNING**

Production failures become new Golden Set cases and recalibrated sensors.

# Welcome to the Laboratory

Scrum was designed for uncertainty in requirements. AI adds uncertainty in behavior, so engineering rigor must move from deterministic delivery control into empirical measurement and adaptation.

## DETERMINISTIC SOFTWARE FACTORY

### Predict → Build → Verify → Ship

The process assumes that the same input and code path produce the same output. Variation is treated as defect leakage.



**Factory metric:** conformance to specification under repeatable conditions.

## OLD RIGOR

Control the delivery pipeline so the output is exactly reproducible.

## NEW RIGOR

Control the experiment loop so variation is measured, bounded, and corrected.

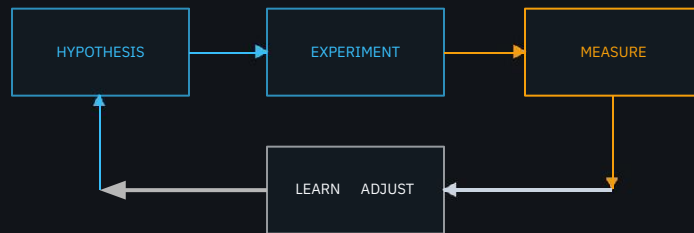
## TEAM IMPLICATION

Scrum ceremonies become checkpoints for evidence, risk, and behavioral drift.

## EMPIRICAL AI LABORATORY

### Hypothesize → Measure → Adapt

The process assumes behavioral variance. Rigor comes from controlled experiments, Golden Sets, Eval Gates, and feedback loops.



**Laboratory metric:** stability of behavior inside an approved tolerance envelope.

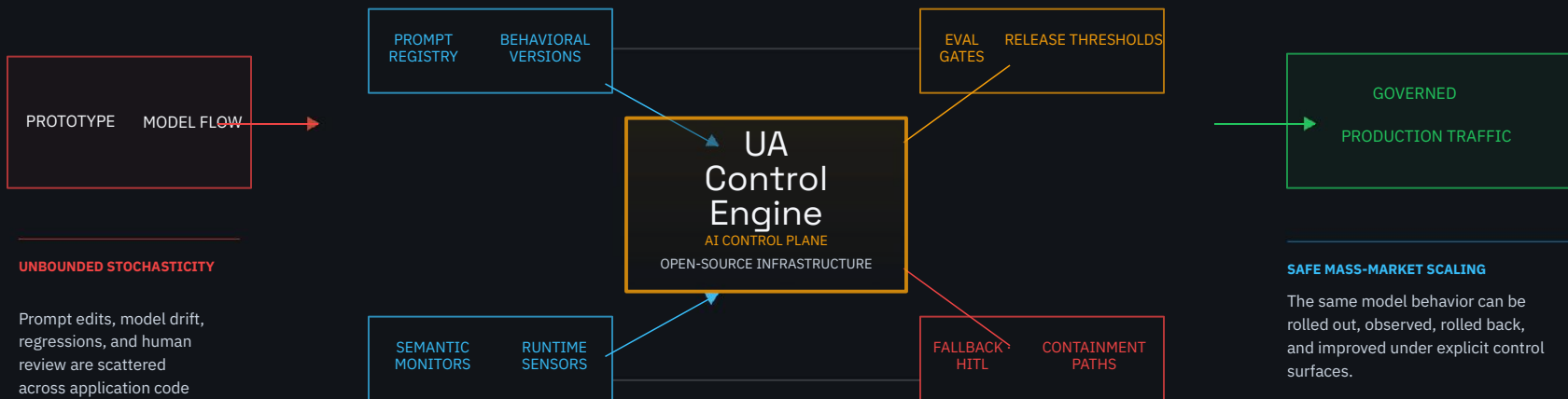
# UA: Open-Source Control Engine

Uncertainty Architecture turns the control patterns in this talk into an open-source AI Control Plane: a composable engine for scaling stochastic systems without surrendering engineering rigor.

BEFORE CONTROL

UNCERTAINTY ARCHITECTURE CONTROL PLANE

AFTER CONTROL



## UNBOUNDED STOCHASTICITY

Prompt edits, model drift, regressions, and human review are scattered across application code and incident playbooks.

## SAFE MASS-MARKET SCALING

The same model behavior can be rolled out, observed, rolled back, and improved under explicit control surfaces.

**Core idea:** KEEP STOCHASTIC CAPABILITY INSIDE A CLOSED-LOOP ARCHITECTURE WHERE BEHAVIOR IS VERSIONED, MEASURED, CONSTRAINED, AND RECOVERABLE.  
COMPOSABLE · AUDITABLE · PORTABLE · RESILIENT

## DISCUSSION

# Q&A

## Time to Build Closed Loops

Non-determinism is not a reason to abandon engineering rigor. It is the reason to move rigor into measurement, feedback, and control.

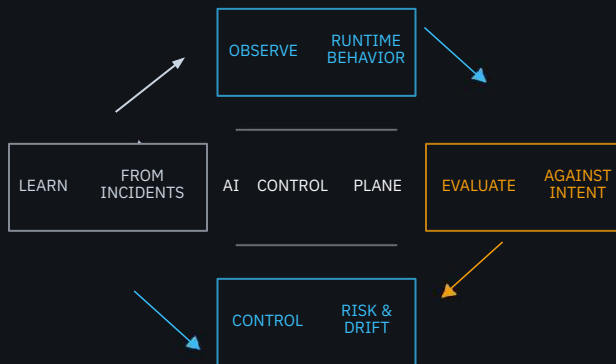


### ENGINEERING RIGOR REMAINS

Requirements become envelopes, tests become sensors, and releases become control decisions.

## FINAL OPERATING MODEL

Design AI systems as feedback-controlled production systems.



The surrounding system continuously steers stochastic behavior toward business intent.

### TEAM ROLES EVOLVE

PM, QA, architects, and engineers share responsibility for the **closed loop**.

## CONTINUE THE CONVERSATION

## Research, links, and contact

### RESEARCH LINK

<https://github.com/UncertaintyArchitectureGroup/uncertainty-architecture>

### CONTACT

Vitalii Oborskyi · [oborskyivitalii@gmail.com](mailto:oborskyivitalii@gmail.com) · <https://www.linkedin.com/in/vitalioborskyi/>

### PROJECT

Uncertainty Architecture · AI Control Plane



Scan Me

### NEXT ACTION

Pick one AI workflow and define its Golden Set, Eval Gate, fallback path, and human escalation policy.